TAD COLA

Práctica 2

Candela Bello González
Izan Arraez Puido
Diego Mora Solano

Fecha de entrega: 26.04.2026

Nº de Grupo de prácticas: 9
Grupo de Laboratorio: 3L (18:00 a 20:00)
Profesora de laboratorio: Nuria Serrano Nieto

ÍNDICE

Especificación lógica completa de los TADs diseñados 2

Dibujo y descripción de los procesos 3

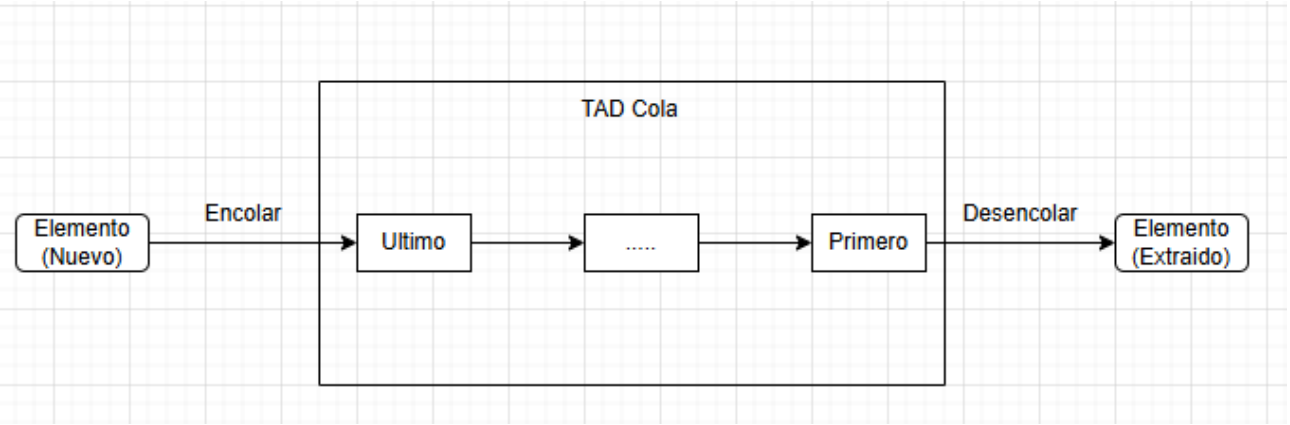
Código fuente desarrollado 3

Pruebas de ejecución 16

Preguntas finales 16

Especificación lógica completa de los TADs diseñados

1. **Definición:** El TAD Cola es una estructura de datos que almacena una colección de elementos bajo el principio de la Colas (FIFO). Es decir, se inserta por el final y se extrae por el principio.



2. **Elementos:** Elementos de cualquier tipo de dato genérico <E>.
3. **Tipo de organización:** Lineal. Los elementos mantienen una secuencia lógica y cronológica estricta en función de su orden de llegada.
4. **Dominio de los elementos del TAD:** El dominio es cualquier secuencia finita de elementos de tipo <E>, cuyo límite viene dado por la memoria disponible en el sistema.
5. **Operaciones básicas**
   - **Nombre:** encolar
     - **Descripción:** Añade un nuevo elemento a la colección situándose en el extremo final de la cola.
     - **Datos de entrada:** Un elemento de tipo genérico E.
     - **Datos de Salida:** Ninguno.
     - **Precondiciones:** Ninguna.
     - **Postcondiciones:** La cola pasa a tener un elemento más. El dato insertado se convierte en el último elemento de la cola.

   - **Nombre:** desencolar
     - **Descripción:** Extrae, elimina y devuelve el elemento que se encuentra al principio de la cola.
     - **Datos de entrada:** Ninguno.
     - **Datos de Salida:** El elemento de tipo E que estaba en el frente. Si está vacía, devuelve null.
     - **Precondiciones:** La cola no debe estar vacía.
     - **Postcondiciones:** La cola pasa a tener un elemento menos. El elemento que ocupaba la segunda posición pasa a ser el nuevo primero.

   - **Nombre:** primero
     - **Descripción:** Devuelve una referencia al elemento situado en la parte frontal de la cola sin eliminarlo.
     - **Datos de entrada:** Ninguno.
     - **Datos de Salida:** El elemento de tipo E del frente. Si está vacía, devuelve null.
     - **Precondiciones:** La cola no debe estar vacía.
     - **Postcondiciones:** El estado y la cantidad de elementos de la cola permanecen inalterados.

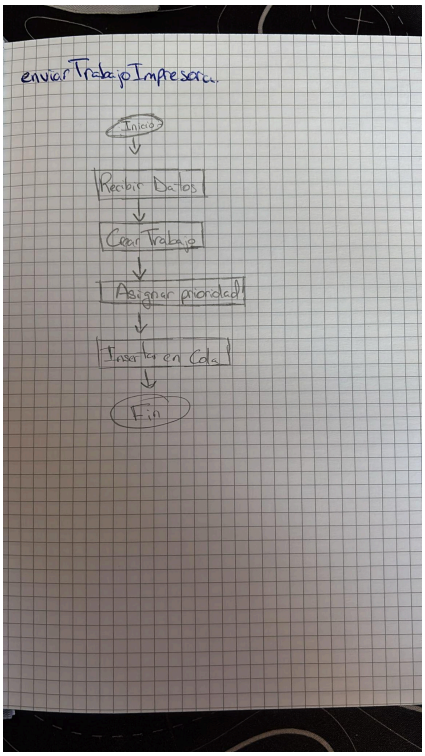
   - **Nombre:** esVacia
     - **Descripción:** Comprueba si la cola contiene o no elementos en su interior.
     - **Datos de entrada:** Ninguno.
     - **Datos de Salida:** true si no hay elementos y false en caso contrario.

- **Precondiciones:** Ninguna.
- **Postcondiciones:** El estado de la cola permanece inalterado.

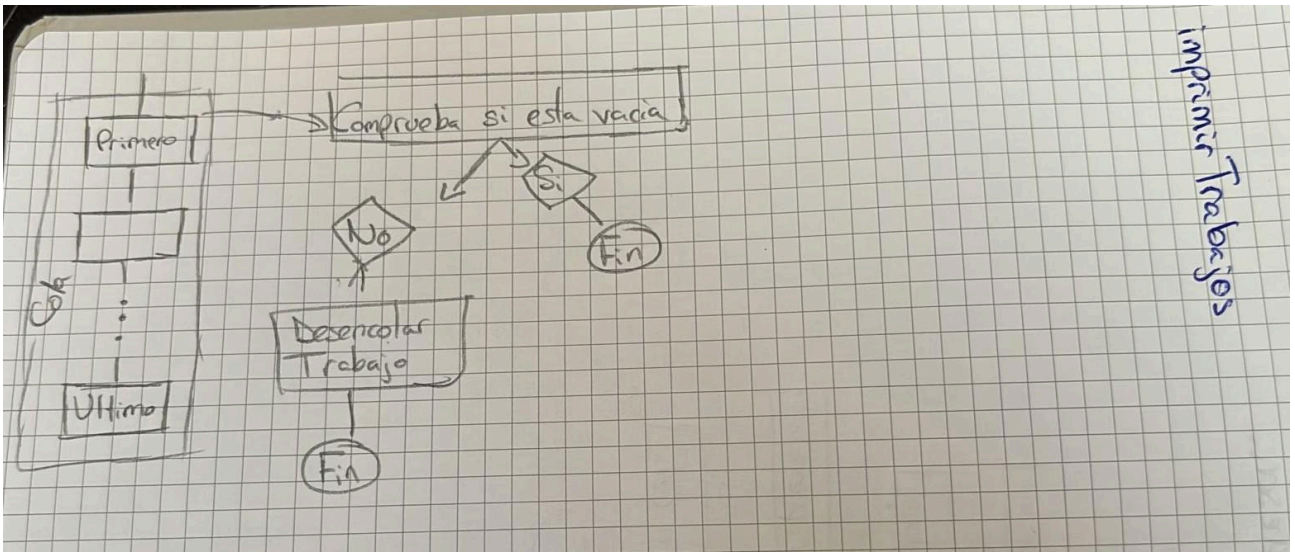
- **Nombre:** contarElemCola
  - **Descripción:** Calcula y devuelve el número total de elementos almacenados en la cola mediante un proceso recursivo.
  - **Datos de entrada:** Ninguno.
  - **Datos de Salida:** Un valor entero que representa la cantidad de elementos.
  - **Precondiciones:** Ninguna.
  - **Postcondiciones:** El estado de la cola permanece inalterado.

Dibujo y descripción de los procesos

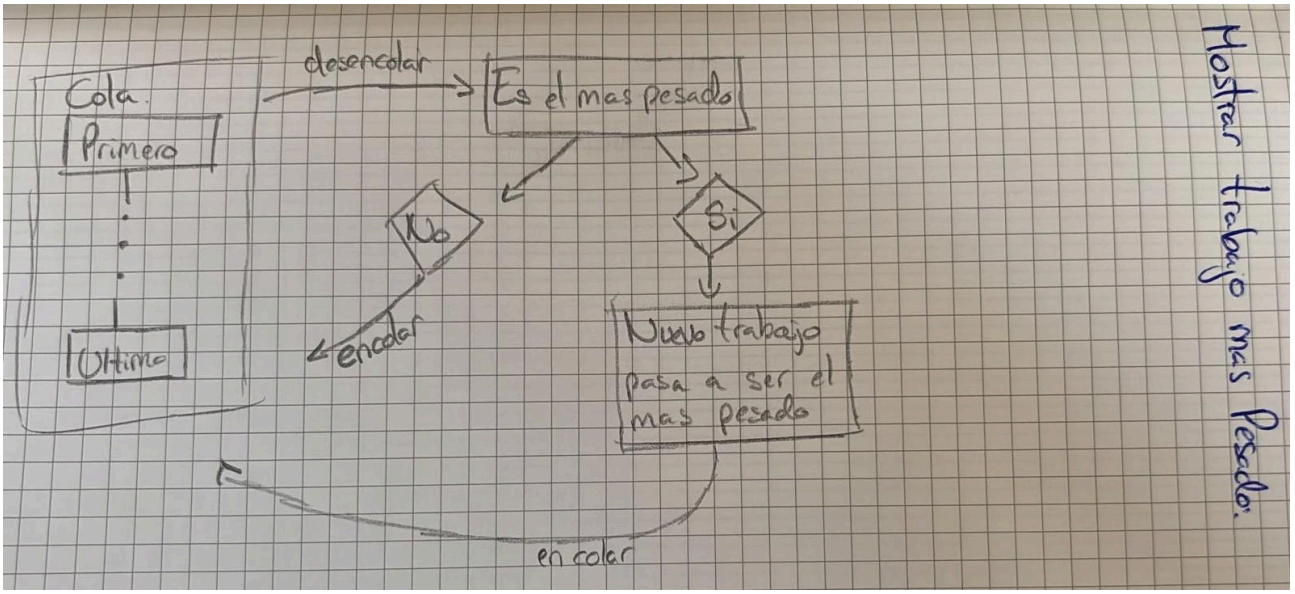
enviaTrabajoImpresora: Recibe los parametros, crea el Trabajo, le asigna una prioridad, lo inserta en la cola.



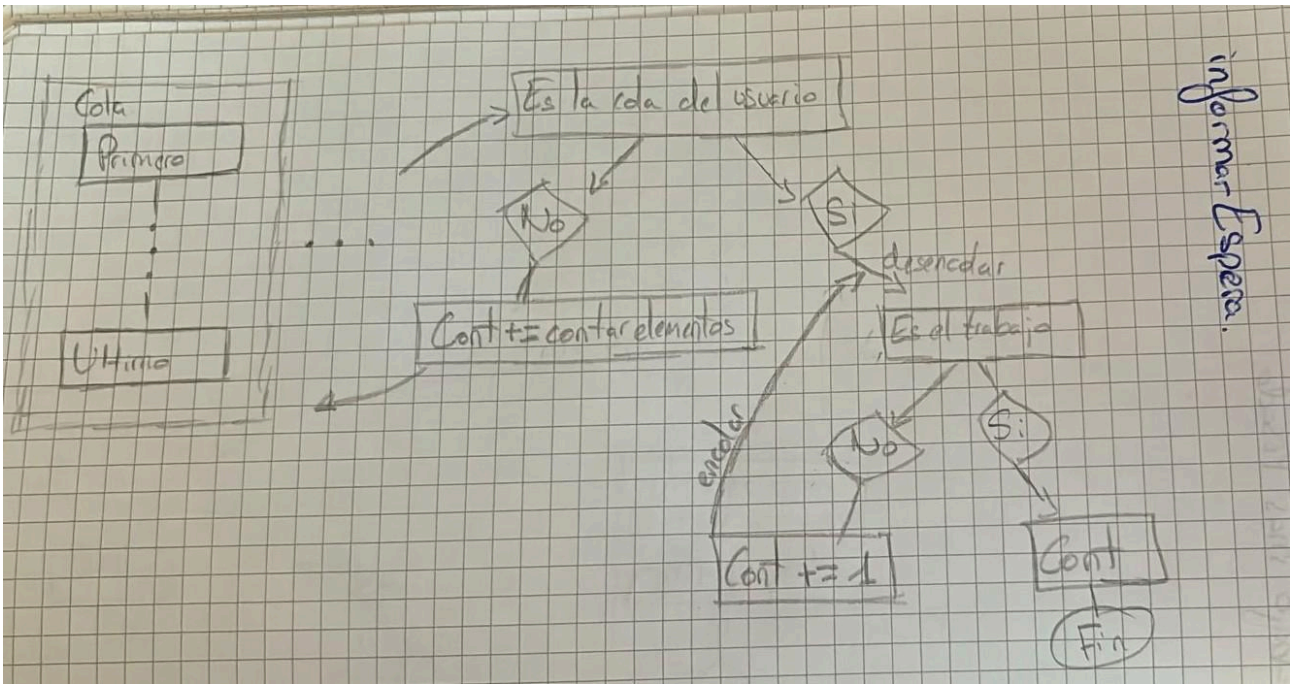
imprimirTrabajos: Busca y extrae el trabajo con mayor prioridad en el sistema para simular su impresion.



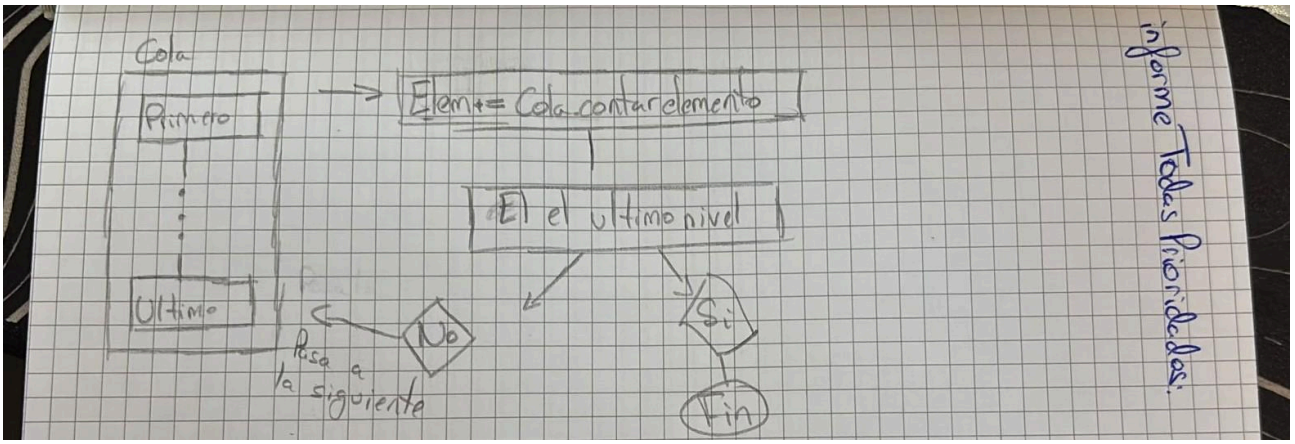
mostrarTrabajoMasPesado: Busca y muestra el trabajo de impresión más pesado que encuentra actualmente en espera en todo el sistema.



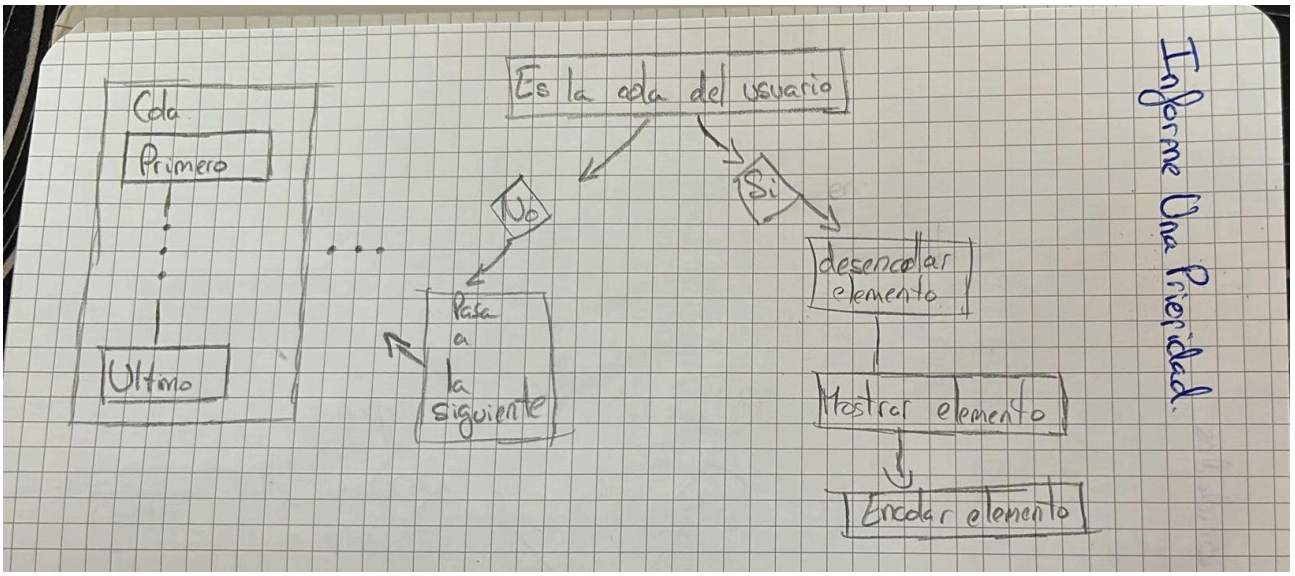
informarEspera: Calcula y muestra la cantidad de trabajos de impresión que un usuario específico tiene por delante antes de que su documento sea impreso.



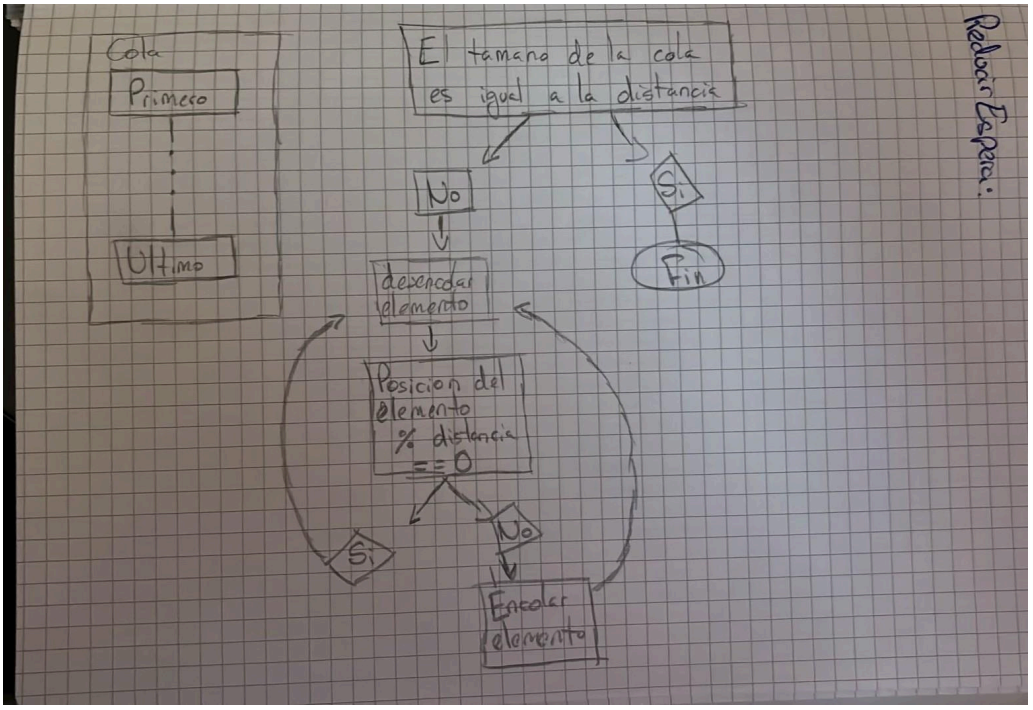
informeTodasPrioridades: Muestra el número de trabajos encolados para cada uno de los 9 niveles de prioridad, calculando a su vez el total global de trabajos pendientes en todo el sistema.



informeUnaPrioridad: Muestra por pantalla todos los trabajos asociados a una cola de prioridad específica, manteniendo su orden de llegada.



reducirEspera: Reduce el tiempo de espera en una cola específica eliminando trabajos.



Código fuente desarrollado

```
package librerias.estructurasDeDatos.lineales;

import librerias.excepcionesDeUsuario.Validaciones;

import java.util.Scanner;

/**
 * Clase principal de la aplicación.
 */
public class Main {
```

```java
private static Scanner sc = new Scanner(System.in);

/**
 * Método principal que arranca la ejecución del programa.
 */
public static void main(String[] args) {

    GestorImpresora gestorImpresora = new GestorImpresora();
    gestorImpresora.rellenarDatosPrueba();

    boolean salida = false;

    do {
        try{
            menuPrincipal();
            int eleccion = Integer.parseInt(sc.nextLine());

            switch (eleccion) {
                case 1:
                    enviarTrabajoImpresora(gestorImpresora);
                    break;
                case 2:
                    imprimirTrabajos(gestorImpresora);
                    break;
                case 3:
                    mostrarTrabajoMasPesado(gestorImpresora);
                    break;
                case 4:
                    mostrarTIempoEspera(gestorImpresora);
                    break;
                case 5:
                    informeTodasPrioridades(gestorImpresora);
                    break;
                case 6:
                    mostrarInformeUnaPrioridad(gestorImpresora);
                    break;
                case 7:
                    mostrarReducirEspera(gestorImpresora);
                    break;
                case 8:
                    reiniciarSistema(gestorImpresora);
                    break;
                case 0:
                    salida = true;
                    break;

            }
        }catch (NumberFormatException e){
            System.out.println("Introduce un numero valido");
        }catch (IllegalArgumentException e){
            System.out.println("Error: "+e.getMessage());
        }
    }while (!salida);
}
```

```java
/**
 * Muestra por consola el Menú Principal con sus 9 opciones disponibles.
 */
private static void menuPrincipal(){

    System.out.println("\nMENU PRINCIPAL");
    System.out.println("--------------");
    System.out.println("1.-Enviar un trabaja a la impresora");
    System.out.println("2.-Imprimir trabajos");
    System.out.println("3.-Mostrar trabajo mas pesado");
    System.out.println("4.-Mostrar tiempo de espera del usuario");
    System.out.println("5.-Informe de trabajos por prioridad");
    System.out.println("6.-Informe de trabajos de una prioridad");
    System.out.println("7.-Reducir espera en una prioridad");
    System.out.println("8.-Reiniciar el Sistema de impresion");
    System.out.println("0.-Salir");
    System.out.print("\nSeleccione una opción: ");
}

/**
 * Solicita al usuario para crear un nuevo trabajo de impresión y
 * Valida el ID del usuario.
 *
 * @param gestorImpresora Instancia del gestor del sistema de colas.
 */
private static void enviarTrabajoImpresora(GestorImpresora gestorImpresora){

    System.out.println("Introduce un id de usuario");
    int idUsuario = Integer.parseInt(sc.nextLine());

    Validaciones.rangosIdUsuario(idUsuario);

    System.out.println("Introduce el titulo del trabajo");
    String titulo = sc.nextLine();

    System.out.println("Introduce el tamaño del del trabajo");
    int peso = Integer.parseInt(sc.nextLine());

    gestorImpresora.enviarTrabajoImpresora(idUsuario, titulo, peso);
}

/**
 * Imprime los trabajos en espera y se detenie
 * si el usuario lo indica o si el sistema se queda sin trabajos.
 *
 * @param gestorImpresora Instancia del gestor del sistema de colas.
 */
private static void imprimirTrabajos(GestorImpresora gestorImpresora){

    String respuesta = "";
    boolean mastrabajos;

    do {
        mastrabajos = gestorImpresora.imprimirTrabajos();
```

```
            if (mastrabajos) {
                System.out.println("¿Desea imprimir otro trabajo?(S/N)");
                respuesta = sc.nextLine().toUpperCase();
            }

        }while (respuesta.equals("S") && mastrabajos);
    }

    /**
     * Invoca al gestor para buscar y mostrar el trabajo de impresión más pesado.
     *
     * @param gestorImpresora Instancia del gestor del sistema de colas.
     */
    private static void mostrarTrabajoMasPesado(GestorImpresora gestorImpresora){
        gestorImpresora.mostrarTrabajoMasPesado();
    }

    /**
     * Invoca al gestor para generar un informe general detallando la cantidad
     * de trabajos pendientes por nivel de prioridad.
     *
     * @param gestorImpresora Instancia del gestor del sistema de colas.
     */
    private static void informeTodasPrioridades(GestorImpresora gestorImpresora){
        gestorImpresora.informeTodasPrioridades();
    }

    /**
     * Solicita al usuario un ID y consulta cuántos trabajos de impresión tiene
     * por delante.
     *
     * @param gestorImpresora Instancia del gestor del sistema de colas.
     */
    private static void mostrarTIempoEspera(GestorImpresora gestorImpresora){
        System.out.println("Introduce un id de usuario");
        int idBusqueda = Integer.parseInt(sc.nextLine());

        Validaciones.rangosIdUsuario(idBusqueda);
        gestorImpresora.informarEspera(idBusqueda);
    }

    /**
     * Solicita al usuario un nivel de prioridad y muestra un informe con todos
     * los trabajos que se encuentran esperando en dicha cola.
     *
     * @param gestorImpresora Instancia del gestor del sistema de colas.
     */
    private static void mostrarInformeUnaPrioridad(GestorImpresora
gestorImpresora){

        System.out.println("Introduce la prioridad que se desea consultar (1 al
9)");
        int prioridadConsulta = Integer.parseInt(sc.nextLine());

        Validaciones.rangoPrioridad(prioridadConsulta);
```

```
        gestorImpresora.informeUnaPrioridad(prioridadConsulta);
    }

    /**
     * Solicita al usuario una prioridad para aplicar el algoritmo de reducción de
     * esperas mediante saltos. Valida la prioridad antes de la ejecución.
     *
     * @param gestorImpresora Instancia del gestor del sistema de colas.
     */
    private static void mostrarReducirEspera(GestorImpresora gestorImpresora){

        System.out.println("Introduce la prioridad para reducir la espera (1 al
9)");
        int espera = Integer.parseInt(sc.nextLine());
        Validaciones.rangoPrioridad(espera);

        gestorImpresora.reducirEspera(espera);
    }

    /**
     * Invoca al gestor para abortar todos los trabajos del sistema, vaciando
     * completamente las colas e imprimiendo el listado de elementos abortados.
     *
     * @param gestorImpresora Instancia del gestor del sistema de colas.
     */
    private static void reiniciarSistema(GestorImpresora gestorImpresora){
        gestorImpresora.reiniciarSistema();
    }

}

package librerias.estructurasDeDatos.lineales;

/**
 * Representa un trabajo de impresión en el sistema.
 */
public class Trabajo {

    private int idUsuario;
    private int prioridad;
    private String titulo;
    private int peso;

    /**
     * Constructor principal, que inicializa un trabajo de impresión y calcula
     * su prioridad automáticamente basándose en el ID del usuario.
     * @param idUsuario Identificador único del usuario.
     * @param titulo    Título descriptivo del documento.
     * @param peso      Tamaño en Kb del documento.
     */
    public Trabajo(int idUsuario, String titulo, int peso) {
        this.idUsuario = idUsuario;
        this.prioridad = asignarPrioridad(idUsuario);
        this.titulo = titulo;
```

```
        this.peso = peso;
    }

    public int getIdUsuario() {
        return idUsuario;
    }

    public int getPrioridad() {
        return prioridad;
    }

    public String getTitulo() {
        return titulo;
    }

    public int getPeso() {
        return peso;
    }

    /**
     * Método de servicio privado que calcula la prioridad del usuario
     * @param idUsuario El identificador del usuario.
     * @return Un número entero del 1 al 9 representando la prioridad asignada.
     */
    private int asignarPrioridad(int idUsuario) {
        return idUsuario / 100;
    }
}

package librerias.estructurasDeDatos.lineales;

/**
 * Representa un nodo genérico individual para una estructura de datos lineal
 * enlazada simple.
 *
 * @param <E> Tipo genérico del dato que almacenará el nodo.
 */
public class NodoLEG<E> {

    private E dato;
    private NodoLEG<E> siguiente;

    /**
     * Constructor del nodo. Inicializa el nodo con un dato y establece
     * el puntero al siguiente nodo como nulo por defecto.
     *
     * @param dato El elemento de tipo genérico E que se guardará en el nodo.
     */
    public NodoLEG(E dato) {
        this(dato, null);
    }

    /**
     * Constructor del nodo. Inicializa el nodo con un dato y establece
     * el puntero al siguiente nodo.
```

```
     *
     * @param dato El elemento de tipo genérico E que se guardará en el nodo.
     * @param siguiente El nodo de tipo genérico E que apunta al siguiente nodo.
     */
    public NodoLEG(E dato, NodoLEG<E> siguiente) {
        this.dato = dato;
        this.siguiente = siguiente;
    }

    //Setter
    public void setDato(E dato) {
        this.dato = dato;
    }

    public void setSiguiente(NodoLEG<E> siguiente) {
        this.siguiente = siguiente;
    }

    //Getters
    public E getDato() {
        return dato;
    }

    public NodoLEG<E> getSiguiente() {
        return siguiente;
    }
}
package librerias.estructurasDeDatos.lineales;

import librerias.estructurasDeDatos.modelos.I_Cola;

/**
 * Implementación del TAD Cola mediante nodos enlazados.
 *
 * @param <E> Tipo genérico de los elementos almacenados.
 */
public class LECola<E> implements I_Cola<E> {

    private NodoLEG<E> primero;
    private NodoLEG<E> ultimo;

    /**
     * Constructor por defecto.
     */
    public LECola(){
        this.primero= null;
        this.ultimo= null;
    }

    /**
     * {@inheritDoc}
     */
    @Override
    public void encolar(E dato) {
        NodoLEG<E> nuevo = new NodoLEG<>(dato);
```

```
    if(esVacia()){
        this.primero = nuevo;
    }
    else {
        this.ultimo.setSiguiente(nuevo);
    }
    this.ultimo = nuevo;
}

/**
 * {@inheritDoc}
 */
@Override
public E desencolar() {
    if (esVacia()) {
        return null;
    }

    E dato = this.primero.getDato();

    this.primero = this.primero.getSiguiente();

    if (esVacia()) {
        ultimo = null;
    }

    return dato;
}

/**
 * {@inheritDoc}
 */
@Override
public E primero() {
    if(esVacia()){
        return null;
    }
    return this.primero.getDato();
}

/**
 * {@inheritDoc}
 */
@Override
public boolean esVacia() {
    return this.primero == null;
}

/**
 * {@inheritDoc}
 * Realiza la llamada al método auxiliar recursivo pasando el primer nodo.
 */
@Override
public int contarElemCola() {
    return contarElemCola(this.primero);
```

```
    }

    /**
     * Método auxiliar privado que calcula el número de nodos de forma recursiva.
     * @param actual El nodo actual que se está evaluando.
     * @return El recuento acumulado de los nodos restantes más el nodo actual.
     */
    private int contarElemCola(NodoLEG<E> actual){
        if (actual == null){
            return 0;
        }
        return 1 + contarElemCola(actual.getSiguiente());
    }
}

package librerias.estructurasDeDatos.lineales;

import librerias.estructurasDeDatos.modelos.I_Cola;

/**
 * Clase que actúa como Gestor del Sistema de Impresión.
 * Administra un array de 9 colas segun su prioridad.
 */
public class GestorImpresora {

    private I_Cola<Trabajo>[] colas;

    /**
     * Constructor de la clase GestorImpresora.
     * Inicializa internamente un array de 9 posiciones, donde cada posición
     * contiene una nueva cola enlazada vacía, representando los
     * 9 niveles de prioridad posibles del sistema.
     */
    public GestorImpresora() {
        this.colas = new LECola[9];
       for(int i = 0; i < 9; i++){
           this.colas[i] = new LECola<Trabajo>();
       }
    }

    /**
     * Crea un nuevo trabajo de impresión y lo inserta en la cola por  su nivel
     * de prioridad.
     * @param idUsuario Identificador numérico del usuario.
     * @param titulo    Cadena de texto con el título del documento a imprimir.
     * @param peso      Tamaño del archivo a imprimir, expresado en Kb.
     */
    public void enviarTrabajoImpresora(int idUsuario, String titulo, int peso){
        Trabajo trabajo = new  Trabajo(idUsuario, titulo, peso);
        colas[trabajo.getPrioridad()-1].encolar(trabajo);
        System.out.println("Trabajo enviado a la impresora");
    }

    /**
     * Busca y extrae el trabajo con mayor prioridad en el sistema
```

```
     * para simular su impresión.
     * @return true si se ha encontrado e impreso un trabajo con éxito o
     * false si todas las colas del sistema se encuentran vacías.
     */
    public boolean imprimirTrabajos(){

        for (int i = 0; i < 9;i++){
            if (!this.colas[i].esVacia()){
                Trabajo trabajo = this.colas[i].desencolar();
                System.out.println("Se ha impreso el trabajo "+trabajo.getTitulo()
                        +" "+("Id Usuario "+trabajo.getIdUsuario()
                        +", Prioridad "+trabajo.getPrioridad()));

                return true;
            }
        }
        System.out.println("No hay mas trabajos para imprimir");
        return false;
    }

    /**
     * Busca y muestra por pantalla el trabajo de impresión más pesado
     * que se encuentra actualmente en espera en todo el sistema.
     * El estado de las colas permanece inalterado tras la ejecución.
     */
    public void mostrarTrabajoMasPesado(){

        Trabajo trabajoMasPesado = null;

        for (I_Cola<Trabajo> cola : this.colas) {
            int elementos = cola.contarElemCola();
            for (int j = 0; j < elementos; j++) {
                Trabajo trabajoActual = cola.desencolar();

                if (trabajoMasPesado == null ||
                        trabajoActual.getPeso() > trabajoMasPesado.getPeso()) {
                    trabajoMasPesado = trabajoActual;
                }
                else if (trabajoActual.getPeso() == trabajoMasPesado.getPeso()) {
                    if (trabajoActual.getPrioridad() <
trabajoMasPesado.getPrioridad() ||
                            trabajoActual.getPrioridad() ==
trabajoMasPesado.getPrioridad()) {
                        trabajoMasPesado = trabajoActual;
                    }
                }
                cola.encolar(trabajoActual);
            }
        }

        if (trabajoMasPesado != null){
            System.out.println("ID_Usuario\tNivel de
Prioridad\tTitulo\t\tTamaño(Kb)");

System.out.println("-----------------------------------------------------");
```

```
            System.out.println(trabajoMasPesado.getIdUsuario()+"\t\t\t\t"
                    +trabajoMasPesado.getPrioridad()+"\t\t\t"
                    +trabajoMasPesado.getTitulo()+"\t"
                    +trabajoMasPesado.getPeso()+"\t");
        }
        else {
            System.out.println("No hay trabajos");
        }
    }

    /**
     * Calcula y muestra la cantidad de trabajos de impresión que un usuario
específico
     * tiene por delante antes de que su documento sea impreso.
     *
     * @param idUsuario El identificador del usuario cuyo tiempo de espera se
desea consultar.
     */
    public void informarEspera(int idUsuario){
        int prioridadUsuario = idUsuario / 100;
        int indiceUsuario = prioridadUsuario - 1;
        int contadorTrabajos = 0;
        boolean encontrado = false;

        for(int i = 0; i< indiceUsuario; i++){
            contadorTrabajos += colas[i].contarElemCola();
        }

        I_Cola<Trabajo> suCola = colas[indiceUsuario];
        int n = suCola.contarElemCola();

        for (int i = 0; i < n; i++) {
            Trabajo t = suCola.desencolar();

            if (!encontrado && t.getIdUsuario() == idUsuario) {
                encontrado = true;
            }

            if (!encontrado){
                contadorTrabajos++;
            }
            suCola.encolar(t);
        }

        if(encontrado){
            System.out.println("** El usuario " +idUsuario +" tiene delante "+
contadorTrabajos + " trabajos");
        }else{
            System.out.println("** No existen trabajos pendientes del usuario
"+idUsuario);
        }
    }

    /**
     * Muestra el número de trabajos encolados para cada uno de los 9 niveles
```

```
     * de prioridad, calculando a su vez el total global de trabajos pendientes
     * en todo el sistema.
     */
    public void informeTodasPrioridades(){
        int totalPendientes = 0;

        System.out.println("\nINFORME DE TRABAJOS POR PRIORIDADES");
        System.out.println("Nivel de Prioridad\tTrabajos en espera");
        System.out.println("--------------------------------------------");

        for(int i= 0; i<colas.length; i++){
            int trabajosEnCola = colas[i].contarElemCola();

            System.out.println("\t\t\t"+(i+1)+"\t\t\t"+trabajosEnCola);
            totalPendientes += trabajosEnCola;
        }

System.out.println("----------------------------------------------------");
        System.out.println("Numero total de trabajos pendientes:
"+totalPendientes);

System.out.println("----------------------------------------------------");
    }

    /**
     * Muestra por pantalla todos los trabajos asociados
     * a una cola de prioridad específica, manteniendo su orden de llegada.
     *
     * @param idUsuario Nivel de prioridad numérico a consultar (1 - 9).
     */
    public void informeUnaPrioridad(int idUsuario){
        I_Cola<Trabajo> colaSeleccionada = colas[idUsuario - 1];
        int n = colaSeleccionada.contarElemCola();

        System.out.println("\nINFORME DE TRABAJOS por Prioridad" + idUsuario);
        System.out.println("ID usuario\t\tTitulo");

System.out.println("----------------------------------------------------------");

        if (n == 0){
            System.out.println("No hay trabajos en espera para esta prioridad");
        }else{
            for(int i = 0; i < n; i++){
                Trabajo t = colaSeleccionada.desencolar();
                System.out.println(t.getIdUsuario()+"\t\t\t"+t.getTitulo());
                colaSeleccionada.encolar(t);
            }
        }
    }

    /**
     * Reduce el tiempo de espera en una cola específica eliminando trabajos
mediante
     * un algoritmo de saltos incrementales.
```

```
     * El proceso finaliza cuando quedan menos trabajos en la cola que el tamaño
del
     * salto actual.
     *
     * @param prioridad El nivel de prioridad de la cola que se va a reducir.
     */
    public void reducirEspera(int prioridad){
        I_Cola<Trabajo> cola = colas[prioridad - 1];
        int distancia = 2;

        while(cola.contarElemCola() >= distancia){
            int n = cola.contarElemCola();
            int contadorPosicion = 1;

            for (int i = 0; i < n; i++) {
                Trabajo t = cola.desencolar();

                if ((contadorPosicion - 1) % distancia == 0) {
                    System.out.println("Se ha eliminado el trabajo " +
t.getIdUsuario() + " <" + t.getTitulo() + "> de la cola");
                }else{
                    cola.encolar(t);
                }
                contadorPosicion++;
            }
            distancia++;
        }
    }

    /**
     * Aborta todos los trabajos pendientes en el sistema, vaciando por completo
     * las 9 colas de prioridad y a medida que se eliminan, se muestra por consola
     * un listado de los trabajos que han sido cancelados.
     */
    public void reiniciarSistema(){
        for(int i = 0; i<colas.length; i++){
            while(!colas[i].esVacia()){
                Trabajo t = colas[i].desencolar();
                System.out.println("Id Usuario: "+t.getIdUsuario()+"\t Titulo:
"+t.getTitulo());
            }
        }
        System.out.println("Sistema de impresion reiniciado (todas las colas
vacias).");
    }

    /**
     * Método auxiliar para realizar pruebas.
     * Carga de forma automatizada un conjunto de trabajos de impresión
predefinidos.
     */
    public void rellenarDatosPrueba() {
        // Formato: idUsuario, titulo, peso
        // Prioridad 1 (IDs 100-199)
        enviarTrabajoImpresora(120, "Examen_Final_Mates.pdf", 450);
```

```
        enviarTrabajoImpresora(150, "Apuntes_Historia.docx", 120);
        enviarTrabajoImpresora(120, "Graficos_Mates_V2.png", 800); // Mismo
usuario (120)

        // Prioridad 2 (IDs 200-299)
        enviarTrabajoImpresora(210, "Presupuesto_Proyecto.xlsx", 50);

        // Prioridad 5 (IDs 500-599) - Para probar el Punto 7 con muchos datos
        enviarTrabajoImpresora(501, "Doc_A.txt", 10);
        enviarTrabajoImpresora(502, "Doc_B.txt", 20);
        enviarTrabajoImpresora(503, "Doc_C.txt", 30);
        enviarTrabajoImpresora(504, "Doc_D.txt", 40);
        enviarTrabajoImpresora(505, "Doc_E.txt", 50);
        enviarTrabajoImpresora(506, "Doc_F.txt", 60);

        // Prioridad 9 (IDs 900-999)
        enviarTrabajoImpresora(999, "Ultima_Hora.pdf", 1500);

        System.out.println("\n[SISTEMA] Se han cargado datos de prueba
automáticamente.");
    }
}


package librerias.estructurasDeDatos.modelos;

/**
 * Interfaz que define la lógica del la Cola.
 *
 * @param <E> Tipo genérico de los elementos que almacenará la cola.
 */
public interface I_Cola<E> {
    /**
     * Inserta un nuevo elemento en la parte final de la cola.
     * @param dato Elemento de tipo genérico E que se desea encolar.
     */
    void encolar(E dato);
    /**
     * Extrae y devuelve el elemento situado en el frente de la cola.
     * @return El elemento de tipo E extraído, o nulo si la cola está vacía.
     */
    E desencolar();
    /**
     * Devuelve el elemento situado en el frente de la cola sin extraerlo.
     * @return El elemento de tipo E del frente, o nulo si la cola está vacía.
     */
    E primero();
    /**
     * Comprueba si la cola no contiene ningún elemento.
     * @return true si la cola está vacía o false en caso contrario.
     */
    boolean esVacia();
    /**
     * Calcula el número total de elementos almacenados actualmente en la cola.
     * @return Un número entero con la cantidad de elementos de la cola.
```

```
     */
    int contarElemCola();
}

package librerias.excepcionesDeUsuario;

/**
 * Clase que centraliza la lógica de validación de los datos
 */
public class Validaciones {

    /**
     * Verifica que el identificador numérico de un usuario se encuentre
     * dentro del rango permitido.
     *
     * @param id El número entero introducido por el usuario.
     * @throws IllegalArgumentException Si el ID es menor a 100 o mayor a 999.
     */
    public static void rangosIdUsuario(int id){
        if (id < 100 || id > 999){
            throw new IllegalArgumentException("El Id de Usuario debe de estar
entre 100 y 999");
        }
    }

    /**
     * Verifica que el nivel de prioridad solicitado se corresponda con un nivel
válido
     * de la estructura.
     *
     * @param prioridad El número de prioridad introducido por el usuario.
     * @throws IllegalArgumentException Si la prioridad es menor a 1 o mayor a 9.
     */
    public static void rangoPrioridad(int prioridad){
        if (prioridad < 1 || prioridad > 9){
            throw new IllegalArgumentException("El Prioridad debe ser entre 1 y
9");
        }
    }
}
```

Pruebas de ejecución

1.-Enviar un trabaja a la impresora

```
Seleccione una opción: 1
Introduce un id de usuario
499
Introduce el titulo del trabajo
ingresos.txt
Introduce el tamaño del del trabajo
20
Trabajo enviado a la impresora
```

Validación

```
Seleccione una opción: 1
Introduce un id de usuario
1000
Error: El Id de Usuario debe de estar entre 100 y 999
```

2.-Imprimir trabajos

```
Seleccione una opción: 2
Se ha impreso el trabajo Examen_Final_Mates.pdf Id Usuario 120, Prioridad 1
¿Desea imprimir otro trabajo?(S/N)
S
Se ha impreso el trabajo Apuntes_Historia.docx Id Usuario 150, Prioridad 1
¿Desea imprimir otro trabajo?(S/N)
S
Se ha impreso el trabajo Graficos_Mates_V2.png Id Usuario 120, Prioridad 1
¿Desea imprimir otro trabajo?(S/N)
S
Se ha impreso el trabajo Presupuesto_Proyecto.xlsx Id Usuario 210, Prioridad 2
¿Desea imprimir otro trabajo?(S/N)
S
Se ha impreso el trabajo ingresos.txt Id Usuario 499, Prioridad 4
¿Desea imprimir otro trabajo?(S/N)
N
```

3.-Mostrar trabajo más pesado

```
Seleccione una opción: 3
ID_Usuario  Nivel de Prioridad  Titulo      Tamaño(Kb)
---------------------------------------------------
999             9           Ultima_Hora.pdf 1500
```

4.-Mostrar tiempo de espera del usuario

```
Seleccione una opción: 4
Introduce un id de usuario
999
** El usuario 999 tiene delante 6 trabajos
```

Validaciones

```
Seleccione una opción: 4
Introduce un id de usuario
1
Error: El Id de Usuario debe de estar entre 100 y 999
```

5.-Informe de trabajos por prioridad

```
Seleccione una opción: 5

INFORME DE TRABAJOS POR PRIORIDADES
Nivel de Prioridad  Trabajos en espera
-------------------------------------------
            1           0
            2           0
            3           0
            4           0
            5           6
            6           0
            7           0
            8           0
            9           1
-----------------------------------------------------
Numero total de trabajos pendientes: 7
-----------------------------------------------------
```

6.-Informe de trabajos de una prioridad

```
Seleccione una opción: 6
Introduce la prioridad que se desea consultar (1 al 9)
5

INFORME DE TRABAJOS por Prioridad5
ID usuario      Titulo
-------------------------------------------------------
501         Doc_A.txt
502         Doc_B.txt
503         Doc_C.txt
504         Doc_D.txt
505         Doc_E.txt
506         Doc_F.txt
```

Validación

```
Seleccione una opción: 6
Introduce la prioridad que se desea consultar (1 al 9)
10
Error: El Prioridad debe ser entre 1 y 9
```

7.-Reducir espera en una prioridad

```
Seleccione una opción: 7
Introduce la prioridad para reducir la espera (1 al 9)
5
Se ha eliminado el trabajo 501 <Doc_A.txt> de la cola
Se ha eliminado el trabajo 503 <Doc_C.txt> de la cola
Se ha eliminado el trabajo 505 <Doc_E.txt> de la cola
Se ha eliminado el trabajo 502 <Doc_B.txt> de la cola
```

Validación

```
Seleccione una opción: 7
Introduce la prioridad para reducir la espera (1 al 9)
10
Error: El Prioridad debe ser entre 1 y 9
```

8.-Reiniciar el Sistema de impresión

```
Seleccione una opción: 8
Id Usuario: 504  Titulo: Doc_D.txt
Id Usuario: 506  Titulo: Doc_F.txt
Id Usuario: 999  Titulo: Ultima_Hora.pdf
Sistema de impresion reiniciado (todas las colas vacias).
```

0.-Salir

```
Seleccione una opción: 0

Process finished with exit code 0
```

Preguntas finales

1. Si hubiésemos llevado a cabo la implementación de lenguaje C, ¿cuál sería el proceso a implementar en la opción 8 del menú? Represéntelo en pseudocódigo. ¿Cuál es la diferencia fundamental respecto a la implementación que hemos realizado en Java?

El proceso a implementar sería que en C debemos guardar temporalmente los nodos para desencolarlos, además imprimir sus datos y luego liberar memoria.

```
MÉTODO reiniciarSistema(ArrayColas colasPrioridad)

    entero i
    Trabajo trabajoAbortado

    PARA i DESDE 0 HASTA 8 HACER

        MIENTRAS NO esVacia(colasPrioridad[i]) HACER

            // 1. Extraemos el elemento
            trabajoAbortado <- desencolar(colasPrioridad[i])
            // 2. Mostramos la información
            IMPRIMIR "Id Usuario: "+trabajoAbortado.getIdUsuario()+"\t Titulo: "+trabajoAbortado.getTitulo()”
            // 3. Liberar la memoria física del nodo extraído
             liberarMemoria(trabajoAbortado)

        FIN MIENTRAS

    FIN PARA
    IMPRIMIR "Sistema de impresión reiniciado (todas las colas vacías)."
```

FIN MÉTODO

La diferencia fundamental respecto con java es el recolector de basura, ya que cuando un nodo se queda huérfano este lo borra, mientras que en C habría que liberar la memoria de forma manual con free

2. Si consideramos la implementación del TAD Cola realizada en clase de teoría mediante la clase ArrayCola… ¿Qué diferencias existen entre la aplicación implementada en esta práctica y la que habríamos realizado si hubiésemos utilizado la clase ArrayCola?.

Las principales diferencias que existen son:

- En la instanciación del constructor de GestorImpresora como this.colas[i] = new ArraCola<Trabajo>(), en vez de this.colas[i] = new LECola<Trabajo>(), ya que la lógica es la misma al implementar la interfaz I_Colas<E>
- En el uso de memoria, ya que con ArrayCola habría que reservar memoria aunque no se utilice, dando como resultado un número máximo de trabajos que se pueden guardar, mientras que con las listas enlazadas el consumo de memoria depende del número de trabajos que hay y el único límite que tiene es la memoria del sistema.
- En la inserción y extracción, ya que en el ArrayCola habría que mover las posiciones del array cada vez que se extraiga un trabajo, para respetar la jerarquía de la cola.